

CANDY DIALOGUE ENGINE

Game State Tracker

1.0.0

A **Game State Tracker** is a variable your game uses to keep track of what the player or the game is currently doing.

Common examples include:

- "explore"
- "combat"
- "inventory"
- "dialogue"
- "cutscene"

At any given moment, your game logic may behave differently depending on this value. For example, movement might be enabled only in "explore", or enemies may attack only during "combat".

Some games also use a second variable to remember the previous state, allowing temporary state changes. For example, when the player opens the inventory during exploration, the game switches to "inventory". When the inventory closes, the game returns to "explore" and not "combat".

Candy DE doesn't enforce any specific game state system, but it provides a small helper function you can adapt: [change_game_state\(\)](#) located in [Candy_Functions.gd](#).

The function is entirely optional, so if you don't use a game state tracking system, you can ignore what follows and you don't need to do anything.

↓↓↓ SCROLL DOWN FOR MORE ↓↓↓

For reference purposes, here's the `change_game_state()` function code, as found in `Candy_Functions.gd`, minus some of the comments and with more or less the same colors we use in our VSCode settings, for readability:

```

/* Change Game State value when dialogue starts:
func change_game_state(caller, method):
    var game_state
    var previous_state

    ## Setup:
    ## Does your game use a game state tracker variable?
    var use_game_state = false

    ## Exit the function if game doesn't use game state tracker:
    if use_game_state == false:
        return

    ## Only change state if called by the main Candy_UI:
    if caller.engine_mode != "UI":
        return

    ## What value represents "dialogue" in your game state tracker?
    var dialogue_state_value = "dialogue"

    ## If your game uses a game state tracker:
    if use_game_state == true:
        ## When dialogue starts:
        if method == "start":
            ## Store previous game state before dialogue starts:
            previous_state = game_state

            ## Change game_state value to indicate dialogue state:
            game_state = dialogue_state_value

        ## When dialogue ends:
        if method == "end":
            ## Revert to previous game state value:
            game_state = previous_state

```

1. The first thing you should do is delete the first two lines You won't need those, they're just placeholders to avoid the script throwing errors:

```

var game_state
var previous_state

```

2. Next, change the line below to 'true' since you want the function to run:

```

## Setup:
## Does your game use a game state tracker variable?
var use_game_state = false

```

3. Skip this line for now, we'll talk about it later:

```

## Only change state if called by the main Candy_UI:
if caller.engine_mode != "UI":
    return

```

4. Now you need to indicate what you want to use as a value for game state when dialogue is running. Just change "dialogue" to anything you want -- you can even change to another variable type (e.g. integer) if that's what your game state logic uses.

```
## What value represents "dialogue" in your game state tracker?  
var dialogue_state_value = "dialogue"
```

5. Now let's look at the logic that actually modifies game state when dialogue begins:

```
## If your game uses a game state tracker:  
if use_game_state == true:  
    ## When dialogue starts:  
    if method == "start":  
        ## Store previous game state before dialogue starts:  
        previous_state = game_state  
  
        ## Change game_state value to indicate dialogue state:  
        game_state = dialogue_state_value
```

You might notice the 'if use_game_state == true' check is redundant, since earlier the code returns if use_game_state is 'false'. It's just an extra precaution in case some users modify the function extensively, and it doesn't impact performance anyway. But feel free to remove the 'if' check if you prefer to always optimize code.

The part you need to edit, now is this:

```
        ## Store previous game state before dialogue starts:  
        previous_state = game_state  
  
        ## Change game_state value to indicate dialogue state:  
        game_state = dialogue_state_value
```

- **If your game state logic uses a variable to record the last/previous game state:**

Change 'previous_state' and 'game_state' in the code to the variables you use for recording the previous state and the current state. For example, if you normally record the current game state to 'global.current_state', change 'game_state' to that.

- **If your game doesn't record the last/previous game state to a variable:**

Remove the first line ('previous_state = game_state'), then replace 'game_state' to the variable you use to record the current game state.

6. And finally, let's look at the part that modifies game state when dialogue ends:

```
## When dialogue ends:  
if method == "end":  
    ## Revert to previous game state value:  
    game_state = previous_state
```

- **If your game state logic uses a variable to record the last/previous game state:**

This is the same as before: replace 'game_state' and 'previous_state' with the variables you actually use in your game state logic.

- **If your game doesn't record the last/previous game state to a variable:**

You'll need to write your own code to replace the line 'game_state = previous_state'. The goal is to write code that updates game state to whatever it needs to be when dialogue ends.

If your game always returns to the same game state when dialogue ends (e.g. it always returns to "game"), that's easy: just write:

```
## When dialogue ends:
if method == "end":
    ## Revert to previous game state value:
    game_state = "game"
```

Unfortunately, if your game can return to different game state values, when dialogue ends, depending on the circumstances, we can't really provide instructions since the solution you'll need is specific to how your game implements game state.

Special Case: Multiple Instances

Remember we told you to ignore step 3 and we'd discuss it later? This is it. Let's look at the code again:

```
## Only change state if called by the main Candy_UI:
if caller.engine_mode != "UI":
    return
```

In Candy DE scripts, 'caller' is the dialogue engine instance that calls a function, instantiates a node/scene, or interacts with a node/scene. Quick rundown:

- Anytime a node has Candy_Engine.gd attached, that's an instance of the dialogue engine. Candy_UI is such an instance, but you can create as many as you want.
- Each engine instance works separately from the others. It runs its own dialogues, has its own settings variables, and typically has its own UI elements (e.g. dialogue box, media players, etc.).
- You can use different instances for different purposes. For example, the recommended method for NPC barks is to assign each NPC node their own dialogue engine instance. This allows them to speak simultaneously in the background, without interfering with each other.
- You can also use different instances when you need different settings for Candy_Engine.gd. For example, in an RPG, you could have one instance for dialogues in 'exploration' mode, and another instance for dialogues during combat.

Therefore, the 'caller' variable in Candy DE scripts is a direct reference to the engine instance node.

'engine_mode' is a variable that can be used to designate the purpose of a dialogue engine instance. For example, Candy_UI has it set to "UI", while Bark Scenes have it set to "Bark". For custom engine instances, you can set any value you like.

By default, in the [change_game_state\(\)](#) function, an engine instance can only modify game state if its engine_mode is "UI". This already filters engine instances that shouldn't change game state, like Barks.

If you create your own engine instances, we recommend giving them their own **engine_mode** value instead of re-using "UI". If these instances should modify game state, you'll need to update the function code to include them.

To make sure the function doesn't return, just change the code a little bit.

Replace:

```
## Only change state if called by the main Candy_UI:
if caller.engine_mode != "UI":
    return
```

With:

```
## Only change state if called by the main Candy_UI:
var allowed_modes = ["UI"]
if caller.engine_mode not in allowed_modes:
    return
```

Then add any modes that should be able to change game state to the **allowed_modes** array.

Finally, you'll need to modify the end of the function, where it modifies game state:

```
## If your game uses a game state tracker:
if use_game_state == true:
    ## When dialogue starts:
    if method == "start":
        ## Store previous game state before dialogue starts:
        previous_state = game_state

        ## Change game_state value to indicate dialogue state:
        game_state = dialogue_state_value

    ## When dialogue ends:
    if method == "end":
        ## Revert to previous game state value:
        game_state = previous_state
```

We recommend using a match statement, with logic for each engine_mode value.

For example, when dialogue ends, if caller.engine_mode is "UI", the function could set game state to "Exploration"; but if engine_mode is "Combat" (e.g. a special dialogue engine instance is used during combat), then the code for that value could change game state to "Combat".

With that said, if all engine_modes that are allowed to change game state use the exact same logic, then you don't need such a match statement: the code above is fine.